Grado en Ingeniería Informática

Modelado y diseño avanzado de software

Integrantes del Grupo:

- Javier Castilla Tejeda
- Rafael Damián Cristea

Enlace al repositorio GitHub:

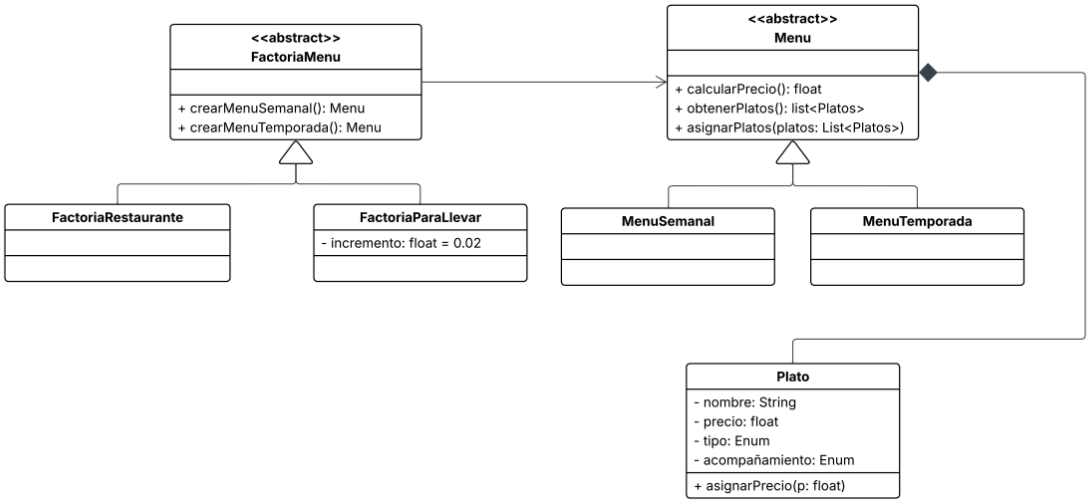
- https://github.com/JavCast03/MDAS_Equipo8

Índice

1. Patrón Abstract Factory **3**
1.1 Diagrama de clases de la solución propuesta 3
1.2 Decisiones de diseño adoptadas 3
2. Patrón Adapter **4**
2.1 Diagrama de clases de la solución propuesta 4
2.2 Decisiones de diseño adoptadas 4
3. Patrón Bridge **5**
3.1 Diagrama de clases de la solución propuesta 5
3.2 Decisiones de diseño adoptadas **5**
4. Patrón Composite **6**
4.1 Diagrama de clases de la solución propuesta 6
4.2 Decisiones de diseño adoptadas **6**
5. Patrón Façade **7**
5.1 Diagrama de clases de la solución propuesta 7
5.2 Decisiones de diseño adoptadas 7

1. Patrón Abstract Factory

1.1 Diagrama de clases de la solución propuesta



1.2 Decisiones de diseño adoptadas

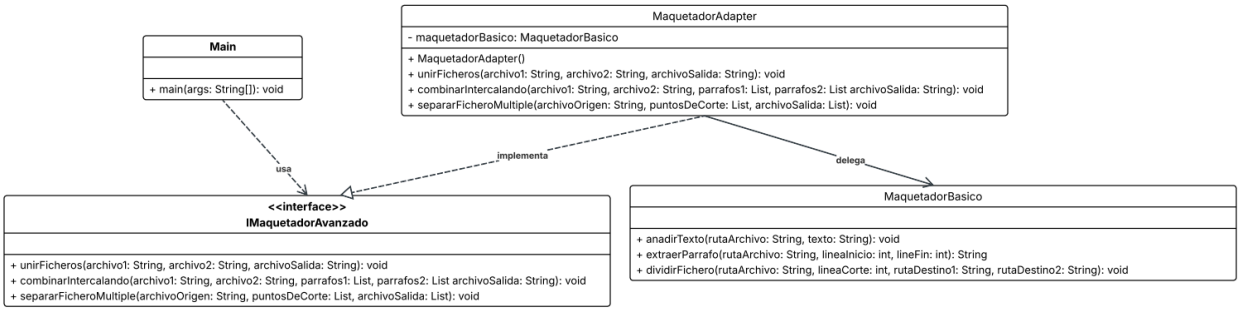
Para la implementación del sistema de pedidos del restaurante, se ha seleccionado el patrón creacional Abstract Factory. La motivación principal ha sido la necesidad de crear y agrupar familias de productos relacionados que varían su lógica de negocio dependiendo de su modalidad de consumo garantizando la consistencia del pedido.

Las decisiones de diseño clave han sido las siguientes:

- **Abstracción de la creación:** Se ha definido la interfaz PedidoFactory con las implementaciones concretas RestauranteFactory y LlevarFactory. Esto aísla al código cliente (Main) de las clases concretas, permitiéndole solicitar productos sin conocer los detalles específicos de su instanciación ni la modalidad exacta tras ser configurada la factoría.
- **Familias de Productos:** Se han creado dos interfaces para los productos abstractos (PlatoMenu y PlatoTemporada). Cada una cuenta con variantes concretas para llevar y para consumir en el local.
- **Encapsulación de reglas de negocio:** Las condiciones del enunciado se han encapsulado directamente en la creación y comportamiento de los productos concretos. La exclusión del postre para los pedidos a domicilio se gestiona descartando el parámetro en LlevarFactory y en el constructor de PlatoMenuLlevar. Asimismo, el incremento del 2% por el servicio a domicilio se aplica directamente en el método calcularPrecio() de los productos PlatoMenuLlevar y PlatoTemporadaLlevar, manteniendo los productos de restaurante con su precio base.

2. Patrón Adapter

2.1 Diagrama de clases de la solución propuesta



2.2 Decisiones de diseño adoptadas

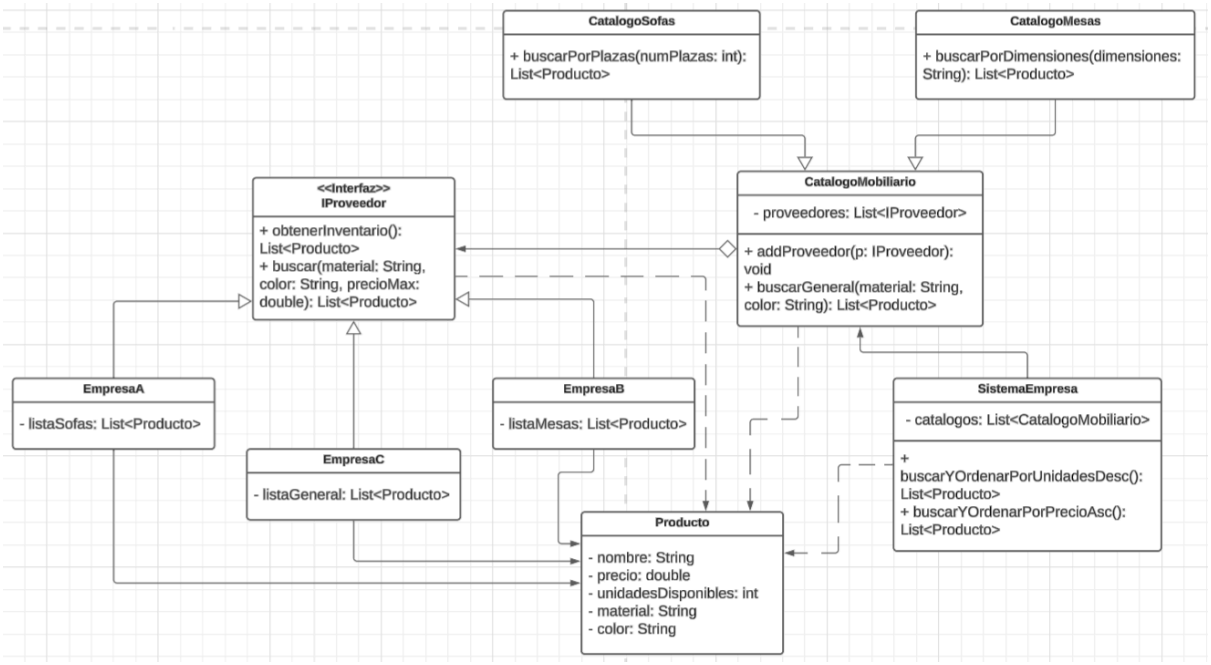
Para resolver el problema del sistema de maquetación, se ha implementado el patrón estructural Adapter. El objetivo principal era adaptar una clase existente que no podía ser modificada, para que cumpliera con una nueva interfaz requerida por el cliente.

La clase MaquetadorAdapter implementa la interfaz requerida y mantiene una relación de composición con MaquetadorBasico. Las adaptaciones lógicas realizadas son las siguientes:

- En la operación unirFicheros, dada la restricción de que el Adapter solo extrae texto mediante líneas de inicio y fin, se ha utilizado el valor Integer.MAX_VALUE como cota superior para garantizar la extracción completa del contenido sin alterar la firma original.
- En la operación separarFicheroMultiple, puesto que el método original solo permite realizar cortes binarios, se ha implementado un algoritmo iterativo. Este proceso divide recursivamente el fragmento restante actualizando el punto de corte relativo, y gestiona dinámicamente la memoria secundaria eliminando los archivos temporales generados durante las iteraciones intermedias.

3. Patrón Bridge

3.1 Diagrama de clases de la solución propuesta



3.2 Decisiones de diseño adoptadas

Para el sistema de catálogos de la empresa de muebles, se ha empleado el patrón estructural Bridge. El problema radica en la fuerte rigidez y la explosión combinatoria de subclases que sufriría el sistema si se intentara acoplar cada categoría de mobiliario con cada proveedor específico (creando, por ejemplo, clases como CatalogoMesasEmpresaA, CatalogoMesasEmpresaB, etc.).

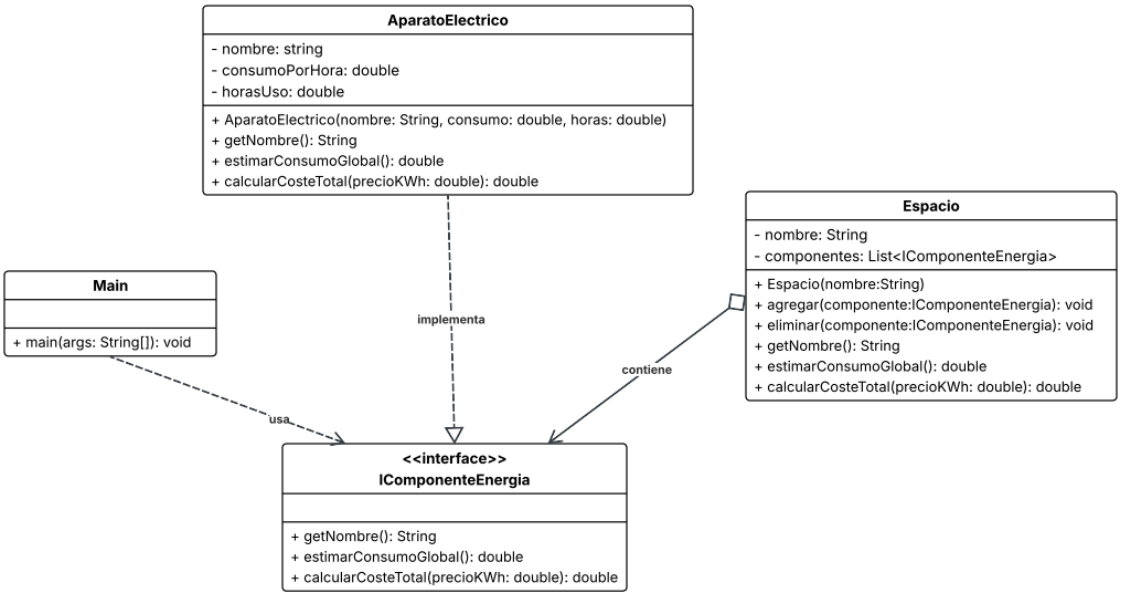
Se ha desacoplado la lógica en dos jerarquías independientes conectadas a través de composición (el "puente"). Este diseño:

- Define una jerarquía de Abstracción (CatalogoMobiliario, refinada por CatalogoSofas y CatalogoMesas) que expone los métodos de búsqueda lógicos para el cliente (buscar por plazas o dimensiones).
- Define una jerarquía de Implementación (la interfaz IProveedor y las concretas EmpresaA, EmpresaB y EmpresaC) que encapsula y estandariza el acceso al inventario físico de los suministradores de terceros.

La decisión clave de diseño ha sido dotar a la abstracción base (CatalogoMobiliario) de una lista de interfaces IProveedor. Esto le permite iterar y delegar de forma dinámica la recuperación de datos en bruto (obtenerInventario()), para luego aplicar sus propios filtros. Además, se ha optado por utilizar una única clase de transferencia de datos genérica (Producto) en lugar de heredar muebles concretos; esto evita que el cliente (SistemaEmpresa) tenga que realizar complejas comprobaciones de tipos o conversiones al agregar y ordenar los inventarios de aquellas empresas que venden múltiples categorías.

4. Patrón Composite

4.1 Diagrama de clases de la solución propuesta



4.2 Decisiones de diseño adoptadas

Para modelar el sistema de control de gasto energético de la universidad, se ha aplicado el patrón estructural Composite, dado que la jerarquía del problema requería tratar de forma uniforme tanto a los elementos individuales como a las agrupaciones de los mismos.

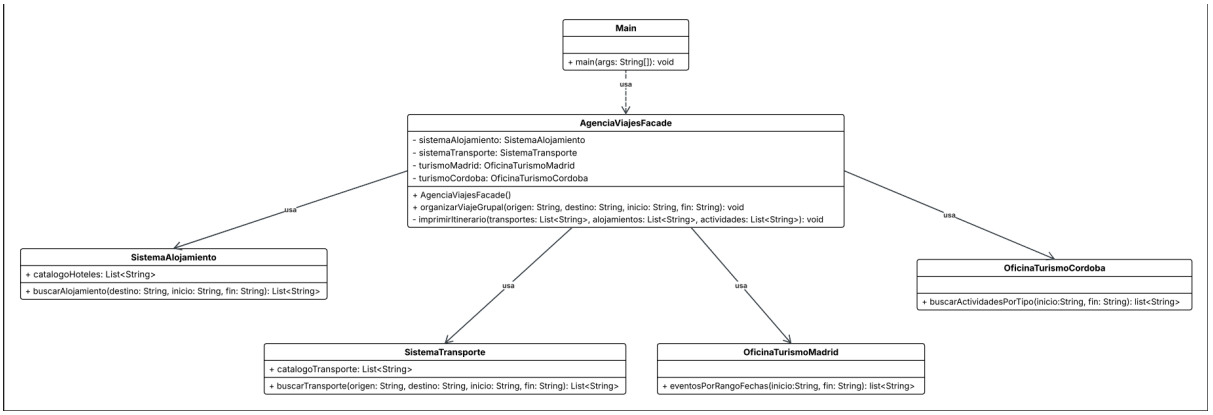
- **Componente:** Se ha diseñado la interfaz IComponenteEnergia, que define las operaciones comunes estimarConsumoGlobal() y calcularCosteTotal().
- **Hoja:** La clase AparatoElectrico implementa el componente calculando su consumo base. No tiene responsabilidades de agregación.
- **Compuesto:** La clase Espacio almacena una colección de IComponenteEnergia.

La principal decisión de diseño ha sido delegar el cálculo del consumo recursivamente. Cuando se solicita el gasto a un Espacio, este itera sobre su colección interna y suma sus resultados. Esto permite al cliente interactuar con un árbol de objetos de profundidad variable sin necesidad de conocer su estructura interna ni distinguir entre un nodo intermedio y una hoja.

---

5. Patrón Façade

5.1 Diagrama de clases de la solución propuesta



5.2 Decisiones de diseño adoptadas

Para el sistema de la agencia de viajes, se ha empleado el patrón estructural Façade. El problema radica en la alta dependencia y el fuerte acoplamiento que sufriría el cliente si tuviera que interactuar directamente con múltiples subsistemas heterogéneos, especialmente con las distintas oficinas de turismo que exigen parámetros incompatibles entre sí.

Se ha introducido la clase AgenciaViajesFacade como un punto de entrada único. Esta clase:

1. Encapsula la instanciación y gestión de los subsistemas subyacentes (SistemaAlojamiento, SistemaTransporte, OficinaTurismoMadrid y OficinaTurismoCordoba).
2. Expone un único método de alto nivel (organizarViajeGrupal) que recibe parámetros primitivos simples (origen, destino, fechas).

La decisión clave de diseño ha sido dotar a la Fachada de la lógica condicional necesaria para orquestar las llamadas a la API correcta de turismo según la ciudad de destino. Por ejemplo, transforma la petición estándar de la agencia en una búsqueda por rango de fechas para Madrid, y en una búsqueda por "tipo de evento" para Córdoba. De este modo, el Cliente queda totalmente aislado de la complejidad estructural e implementativa de los servicios de terceros.